

# CampusCare

## Software Requirements Specification

*A Mobile Application for Facility Management*

---

### Team: [Secret]

Omar Khaled | 13002892 (Team Leader)

Farida Adham | 14001734

Ziad Deifallah | 13006806

Omar Ahmed Ismail | 13006148

Marwan Ibrahim | 14003376

Tutorial: T-01 | TA: Moamen Atia

INCS 617 — Software Engineering for Business Informatics SS 2026

German International University

# 1. Introduction

---

## 1.1 Purpose

This document describes the Software Requirements Specification (SRS) for the CampusCare mobile application. It defines what the system should do and how it should work. It serves as the reference for the development team, the TA, and the instructor throughout the project lifecycle.

## 1.2 Overview of the Software

CampusCare is a mobile application that enables students and staff to report maintenance problems on campus. Users can photograph an issue, add a description, choose a location and category, and submit a ticket. The Facility Management team can then view tickets, assign workers, set priorities, and track each issue through to resolution. The system aims to make the reporting process faster, more organised, and fully transparent.

## 1.3 Product Vision & Scope

**Vision:** To create a simple, efficient platform connecting the GIU community with the Facility Management team, making maintenance faster, better organised, and more transparent.

**Scope** — the project includes:

- A mobile application built with React Native (Expo) and TypeScript
- A backend REST API built with Node.js, Express, and TypeScript
- Prisma ORM over a PostgreSQL database
- JWT-based authentication with bcrypt password hashing
- Cloud-based image storage via Supabase Storage (or local uploads directory for development)
- OpenAPI/Swagger documentation for all API endpoints

The system is limited to maintenance and facility issues within the GIU campus only.

## 1.4 Definitions and Acronyms

| Term / Acronym     | Definition                                                                                                                  |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>SRS</b>         | Software Requirements Specification — a document describing functional and non-functional requirements.                     |
| <b>Ticket</b>      | A digital record created by a community member to report a facility issue (photo, description, location, category, status). |
| <b>RBAC</b>        | Role-Based Access Control — permissions granted based on role: Community Member, Worker, Facility Manager, or System Admin. |
| <b>API</b>         | Application Programming Interface — protocols allowing the mobile app to communicate with the backend.                      |
| <b>RESTful API</b> | An API following the REST architectural style, used for all client-server communication in CampusCare.                      |

| Term / Acronym           | Definition                                                                                                                              |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>MVP</b>               | Minimum Viable Product — the core features for the initial release: issue submission, assignment, status tracking, and notifications.   |
| <b>React Native</b>      | Open-source mobile framework for building the CampusCare frontend (iOS & Android).                                                      |
| <b>Expo</b>              | Toolchain built on React Native that simplifies development, testing, and deployment.                                                   |
| <b>Node.js</b>           | JavaScript runtime used to build the CampusCare backend server.                                                                         |
| <b>Express</b>           | Minimal Node.js web framework used to structure API routes and middleware.                                                              |
| <b>TypeScript</b>        | Statically-typed superset of JavaScript used across both the frontend and backend.                                                      |
| <b>Prisma</b>            | ORM (Object-Relational Mapper) used to interact with the PostgreSQL database.                                                           |
| <b>JWT</b>               | JSON Web Token — a compact, URL-safe token standard used for stateless user authentication.                                             |
| <b>bcrypt</b>            | Password hashing library used to securely store user credentials.                                                                       |
| <b>Zod</b>               | TypeScript-first schema validation library used on both client and server for input validation.                                         |
| <b>Zustand</b>           | Lightweight state-management library used in the React Native frontend.                                                                 |
| <b>Axios</b>             | HTTP client used in the frontend to communicate with the backend API.                                                                   |
| <b>Swagger / OpenAPI</b> | API documentation format; exposed at /api/docs on the backend server.                                                                   |
| <b>Supabase Storage</b>  | Cloud-based file storage service used for uploaded images in production.                                                                |
| <b>PostgreSQL</b>        | Relational DBMS storing all CampusCare data (users, tickets, locations, notifications).                                                 |
| <b>ERD</b>               | Entity Relationship Diagram — visual representation of the database schema.                                                             |
| <b>UML</b>               | Unified Modelling Language — standardised diagrams for system behaviour (e.g., Use-Case Diagram).                                       |
| <b>Actor</b>             | Any user or external system interacting with CampusCare: Community Member, Worker, Facility Manager, System Admin, Notification System. |
| <b>Status</b>            | Current state of a ticket: Pending, In Progress, or Resolved.                                                                           |
| <b>Notification</b>      | Automated message sent when a relevant event occurs (submission, assignment, status update).                                            |

## 2. Functional Requirements

### 2.1 Community Member

| ID       | Requirement                                                                                                                   |
|----------|-------------------------------------------------------------------------------------------------------------------------------|
| FR-CM-01 | The system shall allow a Community Member to create an account by providing name, email, and password.                        |
| FR-CM-02 | The system shall authenticate a Community Member using email and password, issuing a JWT upon success.                        |
| FR-CM-03 | The system shall allow a Community Member to submit an issue including: location, photo, category, and description.           |
| FR-CM-04 | The system shall allow a Community Member to add a textual description when submitting an issue.                              |
| FR-CM-05 | The system shall allow a Community Member to view a list of issues they have submitted.                                       |
| FR-CM-06 | The system shall allow a Community Member to log out of the application (invalidating the session).                           |
| FR-CM-07 | The system shall display the current status of a submitted issue (Pending, In Progress, or Resolved) to the Community Member. |
| FR-CM-08 | The system shall send a notification to a Community Member when the status of their submitted issue is updated.               |

### 2.2 Facility Manager

| ID       | Requirement                                                                                            |
|----------|--------------------------------------------------------------------------------------------------------|
| FR-FM-01 | The system shall allow a Facility Manager to create an account.                                        |
| FR-FM-02 | The system shall authenticate a Facility Manager using email and password, issuing a JWT upon success. |
| FR-FM-03 | The system shall allow a Facility Manager to log out of the application.                               |
| FR-FM-04 | The system shall allow a Facility Manager to view all submitted issues in the system.                  |
| FR-FM-05 | The system shall allow a Facility Manager to update the status of an issue.                            |
| FR-FM-06 | The system shall allow a Facility Manager to assign an issue to a specific Worker.                     |
| FR-FM-07 | The system shall allow a Facility Manager to close an issue once it has been resolved.                 |
| FR-FM-08 | The system shall allow a Facility Manager to set the priority of an issue (Low, Medium, or High).      |

### 2.3 Worker

| ID      | Requirement                                                                                    |
|---------|------------------------------------------------------------------------------------------------|
| FR-W-01 | The system shall allow a Worker to create an account.                                          |
| FR-W-02 | The system shall authenticate a Worker using email and password, issuing a JWT upon success.   |
| FR-W-03 | The system shall allow a Worker to log out of the application.                                 |
| FR-W-04 | The system shall allow a Worker to view issues assigned to them.                               |
| FR-W-05 | The system shall allow a Worker to add comments to an issue.                                   |
| FR-W-06 | The system shall allow a Worker to mark an assigned issue as In Progress.                      |
| FR-W-07 | The system shall allow a Worker to upload a completion photo after finishing work on an issue. |
| FR-W-08 | The system shall send a notification to a Worker when a new issue is assigned to them.         |

## 2.4 System Admin

| ID       | Requirement                                                                                          |
|----------|------------------------------------------------------------------------------------------------------|
| FR-SA-01 | The system shall allow a System Admin to log in using their credentials, issuing a JWT upon success. |
| FR-SA-02 | The system shall allow a System Admin to log out of the application.                                 |
| FR-SA-03 | The system shall allow a System Admin to activate or deactivate any user account.                    |
| FR-SA-04 | The system shall allow a System Admin to view all registered users in the system.                    |

## 2.5 User Stories

| ID       | Role             | User Story                                                                                                                                                                   |
|----------|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| US-CM-01 | Community Member | As a Community Member, I should be able to create an account so that I can log in to the application.                                                                        |
| US-CM-02 | Community Member | As a Community Member, I should be able to log in using my email and password so that I can access the application.                                                          |
| US-CM-03 | Community Member | As a Community Member, I should be able to submit an issue with its details (location, photo, and category) so that the problem can be reported accurately.                  |
| US-CM-04 | Community Member | As a Community Member, I should be able to add a description to the issue so that the worker understands the problem clearly.                                                |
| US-CM-05 | Community Member | As a Community Member, I should be able to view my submitted issues so that I can track their status.                                                                        |
| US-CM-06 | Community Member | As a Community Member, I should be able to log out so that my data remains secure on shared devices.                                                                         |
| US-CM-07 | Community Member | As a Community Member, I should be able to view the status of my issue (Pending, In Progress, Resolved) so that I know the progress of my request.                           |
| US-CM-08 | Community Member | As a Community Member, I should be able to receive notifications when the status of my submitted issue changes so that I am kept informed without manually checking the app. |
| US-FM-01 | Facility Manager | As a Facility Manager, I should be able to create an account so that I can log in to the application.                                                                        |
| US-FM-02 | Facility Manager | As a Facility Manager, I should be able to log in using my email and password so that I can access the application.                                                          |
| US-FM-03 | Facility Manager | As a Facility Manager, I should be able to log out so that my data remains secure on shared devices.                                                                         |
| US-FM-04 | Facility Manager | As a Facility Manager, I should be able to view all submitted issues so that I can assign them to workers.                                                                   |
| US-FM-05 | Facility Manager | As a Facility Manager, I should be able to update an issue's status so that the system reflects the current state.                                                           |
| US-FM-06 | Facility Manager | As a Facility Manager, I should be able to assign an issue to a worker so that the worker can start fixing it.                                                               |
| US-FM-07 | Facility Manager | As a Facility Manager, I should be able to close a resolved issue so that the system reflects completion.                                                                    |
| US-FM-08 | Facility Manager | As a Facility Manager, I should be able to set the priority of an issue (Low, Medium, High) so that workers address the most critical problems first.                        |
| US-W-01  | Worker           | As a Worker, I should be able to create an account so that I can log in to the application.                                                                                  |
| US-W-02  | Worker           | As a Worker, I should be able to log in using my email and password so that I can access the application.                                                                    |
| US-W-03  | Worker           | As a Worker, I should be able to log out so that my data remains secure on shared devices.                                                                                   |
| US-W-04  | Worker           | As a Worker, I should be able to view my assigned issues so that I can start working on them.                                                                                |

| ID       | Role         | User Story                                                                                                                                |
|----------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| US-W-05  | Worker       | As a Worker, I should be able to comment on an issue to indicate completion so that the Facility Manager can update its status.           |
| US-W-06  | Worker       | As a Worker, I should be able to mark an issue as In Progress so that the Facility Manager and Community Member are aware work has begun. |
| US-W-07  | Worker       | As a Worker, I should be able to upload a photo after fixing the issue so that the Facility Manager can verify the work.                  |
| US-W-08  | Worker       | As a Worker, I should be able to receive a notification when a new issue is assigned to me so that I can begin promptly.                  |
| US-SA-01 | System Admin | As a System Admin, I should be able to log in so that I can access the application.                                                       |
| US-SA-02 | System Admin | As a System Admin, I should be able to log out so that my data remains secure on shared devices.                                          |
| US-SA-03 | System Admin | As a System Admin, I should be able to activate or deactivate any account so that I can enforce system access rules.                      |
| US-SA-04 | System Admin | As a System Admin, I should be able to view all registered users so that I can manage the system user base.                               |

## 3. Non-Functional Requirements

### 3.1 Performance

| ID     | Requirement                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Response Time: The system shall ensure that API response times are under 3 seconds under normal load conditions.        |
| NFR-02 | Concurrent Users: The system shall support at least 500 simultaneous users without significant performance degradation. |

### 3.2 Usability

| ID     | Requirement                                                                                             |
|--------|---------------------------------------------------------------------------------------------------------|
| NFR-03 | Ease of Use: The system shall provide an intuitive, user-friendly interface navigable without training. |
| NFR-04 | Fast Submission: The system shall allow users to submit a maintenance ticket in under one minute.       |

### 3.3 Security

| ID     | Requirement                                                                                                                                 |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-05 | Authentication: All API endpoints (except login/register) shall require a valid JWT before granting access.                                 |
| NFR-06 | Password Security: The system shall hash all passwords with bcrypt before storing them in the database.                                     |
| NFR-07 | API Security: The system shall implement rate limiting and Zod-based input validation on all endpoints to protect against misuse.           |
| NFR-08 | Secure Communication: All client-server communication shall use HTTPS.                                                                      |
| NFR-09 | RBAC: Access to system features shall be controlled by the user's assigned role (Community Member, Worker, Facility Manager, System Admin). |

### 3.4 Reliability

| ID     | Requirement                                                                                                     |
|--------|-----------------------------------------------------------------------------------------------------------------|
| NFR-10 | System Availability: The system shall be available at least 99% of the time.                                    |
| NFR-11 | Image Upload Stability: The system shall handle image uploads reliably, even under unstable network conditions. |

### 3.5 Backup & Recovery



| ID            | Requirement                                                                            |
|---------------|----------------------------------------------------------------------------------------|
| <b>NFR-12</b> | Data Backup: The system shall perform automated daily backups.                         |
| <b>NFR-13</b> | Recovery Plan: The system shall include a disaster recovery plan to prevent data loss. |

## 4. Constraints

---

| ID   | Requirement                                                                                       |
|------|---------------------------------------------------------------------------------------------------|
| C-01 | The mobile application shall be developed using React Native (Expo) with TypeScript.              |
| C-02 | The backend shall be developed using Node.js with Express and TypeScript, exposing RESTful APIs.  |
| C-03 | The backend shall use Prisma ORM with a PostgreSQL database.                                      |
| C-04 | Authentication shall be implemented using JWT (JSON Web Tokens) with bcrypt for password hashing. |
| C-05 | Input validation shall be implemented using the Zod library on both client and server.            |
| C-06 | Image storage shall use Supabase Storage in production (local uploads directory in development).  |
| C-07 | The project must be completed within the academic semester timeline.                              |
| C-08 | The system is limited to maintenance-related issues within the GIU campus only.                   |

## 5. Technology Stack

---

| Layer                | Technology                                                                         |
|----------------------|------------------------------------------------------------------------------------|
| Mobile (Frontend)    | React Native, Expo, TypeScript, React Navigation, Zustand, Axios, Zod              |
| Backend              | Node.js, Express, TypeScript, Prisma ORM, JWT auth, bcrypt, Zod, Swagger / OpenAPI |
| Database             | PostgreSQL (Supabase, Neon, or local)                                              |
| Image Storage        | Supabase Storage (or local uploads directory for development)                      |
| API Documentation    | Swagger UI exposed at /api/docs                                                    |
| Secure Communication | HTTPS                                                                              |

## 6. Future Scope

In future versions, the system may include:

- Push notifications for real-time updates
- Analytics dashboard for maintenance statistics
- Gamification features to encourage reporting
- AI-based prediction of frequently occurring issues
- Integration with GIU internal university systems

## Appendix A — Changes from Original SRS (Repo Alignment)

The following changes were made to align this SRS with the actual repository implementation at <https://github.com/OmarKhaled06/SE-Project>:

| Area                 | Original SRS                                | Updated (Repo-Aligned)                                           |
|----------------------|---------------------------------------------|------------------------------------------------------------------|
| Backend framework    | Node.js + REST APIs (unspecified framework) | Node.js + Express (explicit)                                     |
| Language             | Not specified                               | TypeScript (both frontend & backend)                             |
| ORM                  | Not mentioned                               | Prisma ORM added (C-03)                                          |
| Authentication       | Not detailed                                | JWT + bcrypt explicitly required (NFR-06, C-04)                  |
| Validation           | Not mentioned                               | Zod validation library added (NFR-07, C-05)                      |
| State management     | Not mentioned                               | Zustand (frontend) added to tech stack                           |
| HTTP client          | Not mentioned                               | Axios (frontend) added to tech stack                             |
| API docs             | Not mentioned                               | Swagger/OpenAPI at /api/docs added to tech stack                 |
| Image storage        | Cloud storage service (generic)             | Supabase Storage (specific, C-06)                                |
| Definitions/Acronyms | 12 terms                                    | Extended to 24 terms covering all repo technologies              |
| Security NFRs        | NFR-05, NFR-06, NFR-07                      | Expanded to 5 security NFRs (05–09); bcrypt & RBAC made explicit |
| Constraints          | 5 constraints                               | Extended to 8 constraints reflecting actual implementation stack |